WEB PROGRAMMING I · LECTURE 6

CSS GRID
& RESPONSIVE DESIGN

From components → to layout systems

RECAP — LECTURE 5

WHAT FLEXBOX GIVES US

- One axis at a time
- Row or column
- Items align and distribute
- Great for components

FLEXBOX — 1 DIRECTION

Nav | Logo | CTA

→ horizontal flow

LIMITATION

CAN YOU BUILD A FULL PAGE LAYOUT WITH ONLY FLEXBOX?

```
/* nested flex trying to build a page */
.page    { display: flex; flex-direction: column; }
.middle { display: flex; flex-direction: row;    }
.main   { display: flex; flex-direction: column; }

/* 3 levels deep. Gets messy fast. */
```

Flexbox nesting becomes brittle at page scale.

THE PROBLEM

THIS LAYOUT REQUIRES

- Rows and columns simultaneously
- Header spanning full width
- Sidebar beside content
- Footer spanning full width

→ Two dimensions at once.

SECTION 1 — CONTEXT SHIFT

We need 2D control.

Rows × Columns

SECTION 2 — GRID INTRO

GRID = 2D LAYOUT SYSTEM

FLEXBOX

1 axis

row or column

GRID

2 axes

rows and columns

ACTIVATION

```
.container {
  display: grid;
}
```

Same pattern as Flexbox.

One property on the parent. Children respond.

MENTAL MODEL

STEP 1

You define the structure first.

STEP 2

Then place items into it.

This is **fundamentally different** from Flexbox. In Flex, items define the flow. In Grid, **you define the map**.

SECTION 3 — GRID STRUCTURE

DEFINING COLUMNS

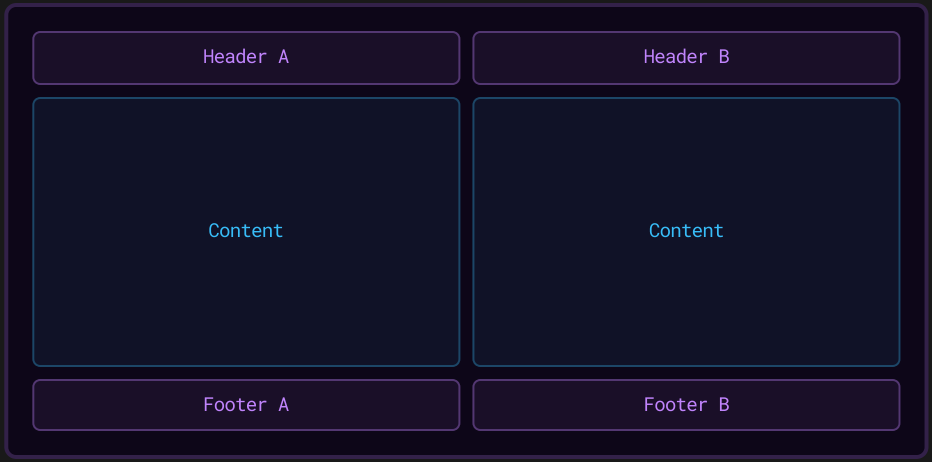
```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}
```

Three equal columns. Items flow left → right, then wrap to the next row.

DEFINING ROWS

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-template-rows: auto 1fr auto;
}
```

auto → shrinks to content
1fr → fills remaining space

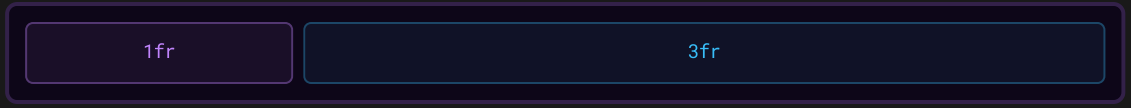


FR

`fr` = FRACTION OF AVAILABLE SPACE

`1fr 1fr 1fr` — EQUAL THIRDS

`1fr 3fr` — SIDEBAR + CONTENT



`200px 1fr` — FIXED + FLEXIBLE

GAP

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 16px;
}

/* or control separately */
row-gap: 24px;
column-gap: 12px;
```

NO GAP

gap: 16px

SECTION 4 — PLACEMENT

DEFAULT PLACEMENT

Items go into cells.

Left → right.

When a row is full, wrap to the next.

Automatically. No extra code.

3 columns → wrap after col 3

GRID-COLUMN: SPAN

```
.wide-item {
  grid-column: span 2;
}

/* or explicit range */
.wide-item {
  grid-column: 1 / 3;
}
```

Item occupies 2 columns instead of 1.

🔥 NAMED AREAS

```
.container {
  display: grid;
  grid-template-columns: 1fr 3fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header  header"
    "sidebar content"
    "footer  footer";
}
```

EACH STRING = ONE ROW

```
"header  header"  ← row 1
"sidebar content"  ← row 2
"footer  footer"  ← row 3
```

The grid map is a visual text representation of your layout.

GRID-AREA

```
/* assign each element to its area */
.header  { grid-area: header;  }
.sidebar { grid-area: sidebar; }
.content { grid-area: content; }
.footer  { grid-area: footer;  }
```

The name connects the element to the region you defined in `grid-template-areas`.

You name

layout regions.

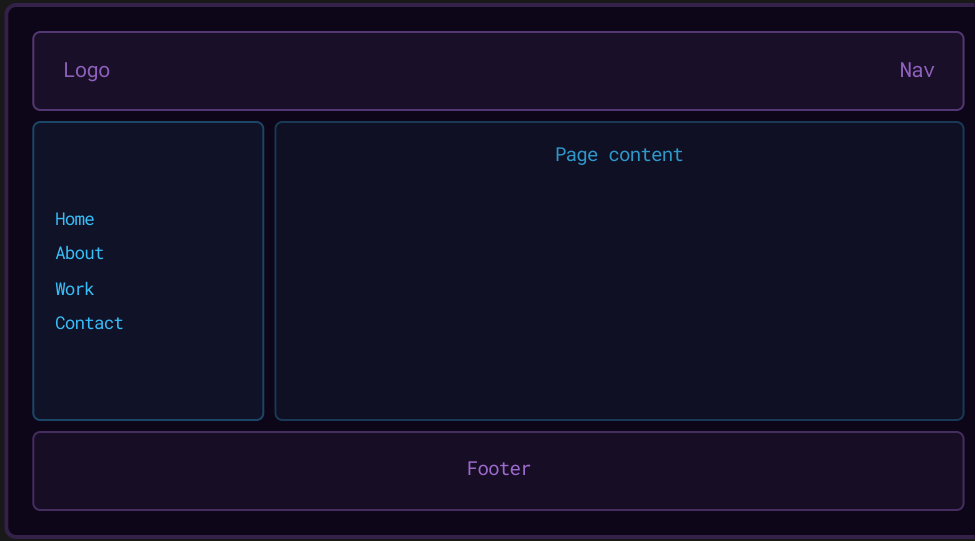
The grid map is separate from the HTML.

Change the layout by editing CSS, not the markup.

SECTION 5 — REAL LAYOUT

```
.page {
  display: grid;
  grid-template-columns: 220px 1fr;
  grid-template-rows: 60px 1fr 50px;
  grid-template-areas:
    "header  header"
    "sidebar content"
    "footer  footer";
  min-height: 100vh;
  gap: 0;
}

header  { grid-area: header;  }
aside   { grid-area: sidebar; }
main    { grid-area: content; }
footer  { grid-area: footer;  }
```



← A real production page layout in ~12 lines of CSS.

FLEXBOX

Component layout

Navbar, cards, buttons

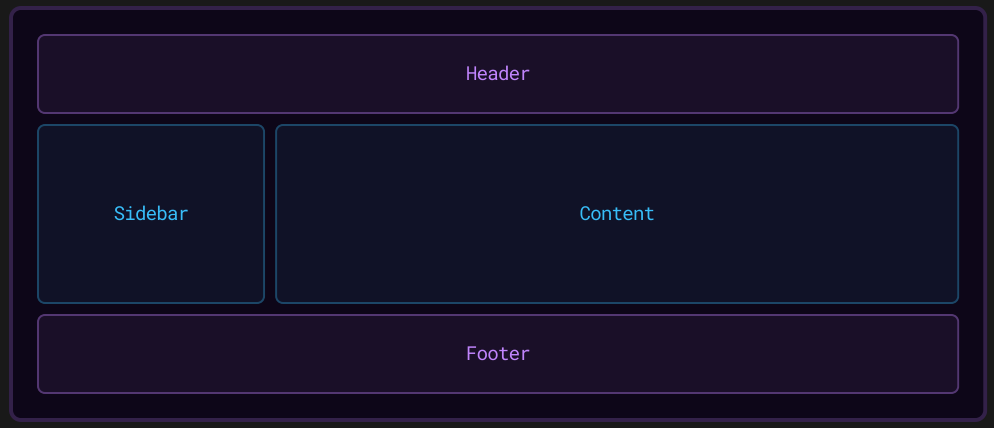
GRID

Page structure

Header, sidebar, footer

Grid = page structure. Flexbox = component layout.

THE PROBLEM



Desktop — 1440px

Mobile — 375px

Different screens. Same layout? That breaks.

SOLUTION

MEDIA QUERIES

Apply CSS rules only when certain conditions are true.

"If the screen is narrower than 768px,
use this different set of styles."

SYNTAX

```
/* This CSS applies to ALL screen sizes */
.container {
  padding: 1rem;
}

/* This ONLY applies when narrower than 768px */
@media (max-width: 768px) {
  .container {
    padding: 0.5rem;
  }
}

/* This ONLY applies when wider than 1024px */
@media (min-width: 1024px) {
  .container {
    max-width: 1200px;
    margin: 0 auto;
  }
}
```

RESULT

LAYOUT CHANGES BASED ON SCREEN SIZE

```
/* desktop: 2 columns */
.page {
  grid-template-columns: 220px 1fr;
  grid-template-areas:
    "header  header"
    "sidebar content"
    "footer  footer";
}

/* mobile: 1 column */
@media (max-width: 768px) {
  .page {
    grid-template-columns: 1fr;
    grid-template-areas:
      "header"
      "content"
      "sidebar"
      "footer";
  }
}
```

DESKTOP

MOBILE

SECTION 7 — MOBILE-FIRST

Start small.

Scale up.

Write mobile styles by default.

Add desktop overrides in media queries.

WHY MOBILE-FIRST?

THE REALITY

- Most web traffic comes from mobile devices
- Small screens = harder constraint
- Solve hard first, simple second
- Less CSS total — override up, not down

THE MINDSET

"What must every user have?"

→ start there.

"What can larger screens afford?"

→ add progressively.

PATTERN

DEFAULT CSS

Mobile layout

No media query needed

`@media (min-width: ...)`

Desktop layout

Added progressively

default = mobile | **@media (min-width) = desktop**

CODE PATTERN

```
/* ── Mobile (default) ──────────── */
.container {
  display: grid;
  grid-template-columns: 1fr;       /* single column */
  grid-template-areas:
    "header"
    "content"
    "sidebar"
    "footer";
}

/* ── Desktop (enhancement) ──────── */
@media (min-width: 768px) {
  .container {
    grid-template-columns: 1fr 3fr; /* sidebar + content */
    grid-template-areas:
      "header  header"
      "sidebar content"
      "footer  footer";
  }
}
```

SECTION 8 — SYSTEM THINKING

You are not styling.

You are designing systems.

THE FULL PICTURE

FLEXBOX

flow

Arrange items along
one axis

GRID

structure

Define page regions
in two dimensions

MEDIA QUERIES

adaptation

Layout responds
to screen context

Three tools. One system. Use all three together.

One layout.

Multiple realities.

One HTML file.

One CSS file.

Every screen. Every device.

CLASSWORK

TASK — RESPONSIVE LAYOUT

Build a page layout that adapts to screen size.

- **Mobile:** single column, stacked
- **Desktop:** sidebar + content, side by side
- Use `grid-template-areas`
- Use mobile-first with `min-width`

When you're done, a user on any device should have a usable, well-structured page.

```
/* ── Mobile (default) ──────── */
.page {
  display: grid;
  grid-template-columns: 1fr;
  grid-template-areas:
    "header"
    "content"
    "sidebar"
    "footer";
}

/* ── Desktop ──────────────── */
@media (min-width: 768px) {
  .page {
    grid-template-columns: 1fr 3fr;
    grid-template-areas:
      "header  header"
      "sidebar content"
      "footer  footer";
  }
}
```